به نام خدا



دانشکده مهندسی کامپیوتر

درس سیستم عامل
نیم‌سال اول ۰۵-۰۶
استاد: دکتر اسدی

تمرین سری دوم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. به سوالات زیر همراه با استدلال علمی پاسخ دهید:

الف) مشخص کنید در هر یک از ۵ سناریوی زیر، پردازه مورد نظر در کدام‌یک از وضعیت‌های اصلی (در حال اجرا، آماده، در انتظار یا خاتمه‌یافته) قرار دارد و در صورت حضور در صف، در کدام صف سیستم‌عامل مستقر است؟ (دلیل هر مورد را به طور خلاصه بنویسید).

۱. پردازه تولیدکننده در مسئله بافر محدود[1] مبتنی بر حافظه مشترک، زمانی که بافر کاملا پر است و شرط حلقه `while (((in + 1) % BUFFER_SIZE) == out)` برقرار است.

۲. پردازه‌ای که بر روی یک لوله بی‌نام در لینوکس که هم‌اکنون ۶۰ کیلوبایت داده خوانده‌نشده درون آن وجود دارد، فراخوانی سیستمی `()write` را برای نوشتن یک پیام ۸ کیلوبایتی به صورت مسدودکننده اجرا کرده است.

۳. پردازه‌ای که فراخوانی سیستمی `()exit` را اجرا کرده، اما پردازه والدش در یک حلقه بی‌نهایت مشغول محاسبات ریاضی است و هنوز فراخوانی `()wait` را انجام نداده است.

۴. پردازه‌ای که بلافاصله پس از رخ دادن یک وقفه تایمر، در مد هسته در حال اجرای دستورالعمل‌های مربوط به ذخیره کردن ثبات‌های پردازنده درون بلوک کنترل پردازه[2] خود است.

ب) در پیاده‌سازی کلاسیک مسئله تولیدکننده‌ـ مصرف‌کننده با استفاده از آرایه حلقوی به اندازه `BUFFER_SIZE` و دو متغیر `in` و `out` (که مقدار اولیه هر دو صفر است)، حداکثر `BUFFER_SIZE - 1` خانه از آرایه قابل استفاده است.

۱. توضیح دهید چرا در آن پیاده‌سازی، اگر اجازه دهیم تمام `BUFFER_SIZE` خانه پر شوند، الگوریتم دچار خطا می‌شود؟

۲. فرض کنید برنامه‌نویسی برای استفاده از تمام ظرفیت آرایه و بدون تعریف متغیر جدید، تنها شرط پر بودن در کد تولیدکننده را از `while (((in + 1) % BUFFER_SIZE) == out)` به `while (in == out)` تغییر دهد. با یک مثال نقض نشان دهید چرا این راه‌حل شکست می‌خورد.

ج) فرض کنید سه پردازه P_1، P_2 و P_3 یک صندوق پستی[3] مشترک به نام A دارند. پردازه P_1 یک پیام به A ارسال می‌کند و هم‌زمان هر دو پردازه P_2 و P_3 دستور `receive(A, msg)` را اجرا می‌کنند.

۱. این سناریو نشان‌دهنده کدام مدل ارتباطی (مستقیم یا غیرمستقیم) است و چه ابهامی در تحویل پیام ایجاد می‌کند؟ چهار راه‌حل معماری که سیستم‌عامل می‌تواند برای رفع این ابهام پیاده‌سازی کند را بنویسید.

۲. در مدل انتقال پیام، مفهوم میعادگاه[4] در چه شرایطی (از نظر مسدودکننده یا غیرمسدودکننده بودن توابع ارسال و دریافت، و ظرفیت بافر لینک ارتباطی) شکل می‌گیرد؟ توضیح دهید چرا در این حالت، پیاده‌سازی مسئله تولیدکننده‌ـ مصرف‌کننده در مقایسه با حافظه مشترک بسیار ساده‌تر است.

و) قطعه کد زیر را در یک سیستم‌عامل مبتنی بر یونیکس در نظر بگیرید. فرض کنید پردازه اولیه (P_0) در لحظه $t = 0$ شروع به اجرا می‌کند، تمامی فراخوانی‌های سیستمی موفق هستند و زمان اجرای دستورات عادی (به‌جز `sleep` و `wait`) ناچیز و صفر است:

```c
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void) {
    for (int i = 1; i <= 3; i++) {
        if (fork() == 0) {
            sleep(i * 2);
```

[1] Bounded-Buffer
[2] PCB
[3] Mailbox
[4] Rendezvous

```
            if (i == 2)
                wait(NULL);
            exit(0);
        }
    }
    sleep(3);
    wait(NULL);
    sleep(4);
    return 0;
}
```

با رسم درخت پردازه‌ها و مشخص کردن زمان دقیق شروع و پایان هر پردازه، تعیین کنید در لحظه $t = 5$ ثانیه:

۱. دقیقا چند پردازه در وضعیت زامبی قرار دارند؟ کدام پردازه‌ها و چرا؟

۲. دقیقا چند پردازه یتیم در سیستم وجود دارند که توسط پردازه `init` بازوالددهی[5] شده باشند (چه در حال اجرا و چه خاتمه‌یافته)؟ چرا؟

۲. فرض کنید برنامه در یک پردازه، تمام فراخوانی‌های سیستمی موفق هستند و هیچ سیگنال خارجی دریافت نمی‌شود. همه‌ی پردازه‌های آماده سرانجام اجرا خواهند شد. هر فراخوانی `printf` را یک رویداد خروجی تجزیه‌ناپذیر در نظر بگیرید، بدین معنی که بدون مسابقه[6] با دستورات `printf` دیگر خروجی چاپ می‌شود. دستور `setvbuf` بافر خروجی را غیرفعال می‌کند. با توجه به تکه کد زیر به سوالات زیر پاسخ دهید.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    int x = 2;
    pid_t p = fork();

    if (p == 0) {
        x += 3;
        if (fork() && fork())
            x += 1;
        else
            x += 2;
        printf("C:%d\n", x);
    } else {
        waitpid(p, NULL, 0);
        printf("P:%d\n", x);
    }
    return 0;
}
```

الف) درخت پردازه‌ها را رسم کنید و تعداد کل پردازه‌ها را با احتساب پردازه‌ی اولیه بنویسید. مشخص کنید هر پردازه توسط کدام فراخوانی `fork` ایجاد شده است.

ب) تمام خطوط چاپ‌شده و تعداد دفعات چاپ هر خط را مشخص کنید. توضیح دهید آیا تغییر مقدار `x` در پردازه‌های فرزند، مقدار آن را در پردازه‌ی اولیه تغییر می‌دهد یا خیر.

[5] Re-parenting
[6] Race

ج) تمام دنباله‌های خروجی متمایز و ممکن را بنویسید. آیا waitpid تضمین می‌کند که خط P:2 آخرین خط خروجی باشد؟ پاسخ خود را توجیه کنید.

د) عملگر && را با || جایگزین کنید و بقیه‌ی کد را تغییر ندهید. درخت جدید پردازه‌ها و خطوط خروجی همراه با تعداد دفعات چاپ آن‌ها را مشخص کنید. در این بخش نیازی به نوشتن تمام دنباله‌های خروجی نیست.

۳. برای این سوال، رفتار بافر خروجی را به این صورت در نظر بگیرید: خروجی برنامه به‌ترتیب وارد یک بافر با ظرفیت ۲۰ بایت می‌شود. وقتی بافر پر شود، علامت پایان خط \n وارد آن شود یا پردازه به‌صورت عادی به پایان برسد، تمام محتوای بافر یک‌جا چاپ می‌شود و بافر خالی می‌شود. فرض کنید که کد زیر بدون هیچ خطایی اجرا می‌شود. خروجی نهایی را با رعایت فاصله‌ها و پایان خط‌ها بنویسید و توضیح دهید چطور به آن رسیده‌اید. چند حالت مختلف خروجی وجود دارد؟

راهنمایی: بافر خروجی بخشی از حافظه‌ی پردازه است.

```c
int main(void) {
    printf("h ");
    pid_t p = fork();
    if (p == 0) {
        printf("b b ");
        pid_t q = fork();
        if (q == 0) {
            printf("b\n");
            return 0;
        }
        waitpid(q, NULL, 0);
        printf("h\n");
        printf("b ");
        return 0;
    }
    printf("b");
    waitpid(p, NULL, 0);
    printf("\n");
    return 0;
}
```

۴. قطعه‌کد زیر را در نظر بگیرید که از حافظه‌ی اشتراکی POSIX و چندین فراخوانی fork() استفاده می‌کند. فرض کنید تمام فراخوانی‌های سیستمی با موفقیت انجام می‌شوند. پردازه‌ها می‌توانند با هر ترتیب زمانی مجازی اجرا شوند و هیچ سازوکار همگام‌سازی دیگری وجود ندارد.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <fcntl.h>

int main() {
    int shm_fd = shm_open("/os_exam_shm", O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(int));
    int *counter = mmap(0, sizeof(int), PROT_READ | PROT_WRITE,
                        MAP_SHARED, shm_fd, 0);
    *counter = 10;

    pid_t p = fork();
    if (p == 0) {
```

```
        *counter += 5;
        if (fork() && fork()) {
            *counter *= 2;
            exit(3);
        } else {
            *counter -= 2;
            exit(7);
        }
    } else {
        int status;
        pid_t child_pid;
        while ((child_pid = wait(&status)) > 0)
            if (WIFEXITED(status))
                *counter += WEXITSTATUS(status);
        printf("Final Result: %d\n", *counter);
    }
    shm_unlink("/os_exam_shm");
    return 0;
}
```

الف) درخت کامل پردازه‌های ایجادشده در طول اجرای برنامه را رسم کنید. رابطه‌ی والد و فرزندی را مشخص کرده و مقدار بازگشتی هر فراخوانی `fork()` را در پردازه‌های مربوط به آن نشان دهید. تعداد کل پردازه‌ها را نیز به دست آورید.

ب) برخلاف متغیرهای محلی معمولی، مقدار اشاره‌شده توسط `counter` در ناحیه‌ای از حافظهٔ اشتراکی POSIX قرار دارد که با `MAP_SHARED` نگاشت شده است. توضیح دهید تغییرات مقدار `*counter` توسط پردازه‌های مختلف چه اثری بر یکدیگر دارند و چرا امکان وقوع مسابقه وجود دارد.

ج) نحوه‌ی خاتمه‌ی هر یک از پردازه‌های فرزند را بررسی کنید. توضیح دهید حلقه‌ی `while` در پردازه‌ی اولیه چگونه با استفاده از `wait()`، `WIFEXITED` و `WEXITSTATUS` وضعیت خروج پردازه‌ها را دریافت می‌کند. آیا این حلقه وضعیت خروج نوه‌ها را نیز دریافت می‌کند؟

د) آیا خروجی دستور زیر در تمام اجراها یکسان است؟ با ارائه‌ی حداقل دو ترتیب اجرای ممکن که خروجی‌های متفاوتی ایجاد می‌کنند، پاسخ خود را توجیه کنید.

```
printf("Final Result: %d\n", *counter);
```

هـ) فرض کنید می‌خواهیم برنامه را به‌گونه‌ای تغییر دهیم که مقدار نهایی `counter` مستقل از ترتیب زمان‌بندی پردازه‌ها باشد. با رعایت شرایط زیر، کد را اصلاح کنید:

- تعداد کل پردازه‌ها همچنان **۴** باشد.
- مقدار اولیه، عملیات محاسباتی و کدهای خروج پردازه‌ها تغییر نکنند.
- استفاده از `mutex`، `semaphore` و ابزارهای مشابه مجاز نیست؛ تنها از `fork()`، `wait()`، `waitpid()` و تغییر ترتیب دستورات استفاده کنید.

ترتیب عملیات روی `counter` باید چنین باشد:

(آ) ابتدا پردازه‌ی C_1 مقدار **۵** را اضافه کند.

(ب) سپس هر یک از دو پردازهٔ نوه مقدار **۲** را کم کند.

(ج) پس از پایان کار هر دو نوه، C_1 مقدار را دو برابر کند.

(د) در پایان، P_0 کد خروج فرزند مستقیم خود را اضافه کند.

توضیح دهید چرا کد اصلاح‌شده مستقل از زمان‌بندی است و مقدار نهایی را محاسبه کنید.

تمارین عملی

۱. در تمرین قبل با سیستم‌عامل xv6 و کلیات آن آشنا شدید، در این تمرین می‌خواهیم با فرایند[7] در این سیستم‌عامل آشنا شویم و با ایجاد تغییرات جزئی و کار با آن، نحوه کار آن در این سیستم‌عامل را بررسی کنیم.

نکته زمانی: در xv6 تایمر تقریباً هر 0.1 ثانیه یک‌بار قطع می‌دهد و باعث می‌شود مقدار ticks یک واحد زیاد شود؛ یعنی pause(10) معادل خوابیدن به مدت یک ثانیه است و تابع uptime() تعداد کل ticks از زمان شروع سیستم را برمی‌گرداند.

۱. برنامه‌ای به نام pingpong بنویسید که با استفاده از fork و دو pipe (یکی برای هر جهت)، یک بایت را بین دو فرایند به صورت رفت‌وبرگشت رد و بدل کند و در پایان مدت انجام کار را گزارش کند. تعداد پیش‌فرض تبادل‌ها 1000 بار باشد. خروجی برنامه باید دقیقاً به این شکل باشد (اعداد مثال هستند):

```
pingpong: 1000 exchanges in 342 ticks
pingpong: 29 exchanges/sec
```

نرخ تبادل در ثانیه را خودتان از روی تعداد ticks محاسبه کنید.

۲. برنامه‌ای به نام race بنویسید که 5 فرایند فرزند بسازد. فرایند فرزند شماره i (از 1 تا 5) باید بعد از 6-i ثانیه خارج (exit) شود و i را به عنوان status برگرداند. سپس فرایند پدر 5 بار منتظر خروج فرزندانش می‌ماند و در هر بار، status برگشتی را چاپ می‌کند. خروجی برنامه باید شامل خطوط زیر باشد (ترتیب خطوط reaped دقیقاً همان ترتیبی است که فرزندان جمع می‌شوند):

```
race: parent reaped child pid 8 with status 5
race: parent reaped child pid 7 with status 4
race: parent reaped child pid 6 with status 3
race: parent reaped child pid 5 with status 2
race: parent reaped child pid 4 with status 1
race: done
```

سؤال ویدیو: چرا فرزندان دقیقاً برعکسِ ترتیب ساخته‌شدن جمع می‌شوند؟ در ویدیو به این سؤال پاسخ دهید.

۳. حالا سراغ هسته بروید. در فایل kernel/proc.h به ساختار struct proc (یعنی PCB هر فرایند) یک فیلد جدید به شکل زیر اضافه کنید:

```
uint64 birth;   // Ticks when this process was created
```

سپس طوری کد بنویسید که این فیلد برای **همه** فرایندها (از جمله اولین فرایند سیستم یعنی init و همه فرایندهایی که با fork ساخته می‌شوند) در لحظه ساخت فرایند و با مقدار ticks جاری مقداردهی اولیه شود. دقت کنید مقدار هر فرایند باید لحظه ساختِ خودِ همان فرایند باشد، نه پدرش.

۴. یک فراخوانی سیستمی جدید به نام getpinfo پیاده‌سازی کنید که یک عکس‌اللحظه[8] از جدول فرایندها را به فضای کاربر منتقل می‌کند:

```
int getpinfo(struct pinfo *arr, int max);
```

ساختار struct pinfo را دقیقاً به شکل زیر در فایل kernel/types.h تعریف کنید تا هم هسته و هم برنامه‌های کاربری آن را ببینند:

[7] process
[8] snapshot

```
struct pinfo {
  int pid;
  int ppid;
  uint64 age;
  char state[10];
  char name[16];
};
```

قرارداد این فراخوانی سیستمی:

- حداکثر max فرایند **فعال** (یعنی state آن‌ها unused نباشد) در آرایه arr پر شود و تعداد واقعیِ پرشده برگردانده شود (در صورت خطا 1-).
- ppid برابر pid والد فرایند باشد؛ اگر والد نداشت (مثل init) مقدار 0 بگیرد.
- age برابر ticks جاری منهای لحظه ساخت فرایند باشد؛ یعنی دقیقاً همان چیزی که در قسمت قبل ساختید.
- state یکی از رشته‌های "used"، "sleeping"، "runnable"، "running" یا "zombie" باشد.
- name نام فرایند (مانند init، sh، ps) باشد.

پیاده‌سازی شما باید هر دو بخش زیر را کامل کند:

- **سمت هسته:** شماره فراخوانی سیستمی، تابع اجرای آن و ثبت در جدول فراخوانی
- **سمت کاربر:** تعریف فراخوانی سیستمی در فضای کاربر و برنامه‌ای به نام ps که خروجی را به صورت جدول زیر چاپ کند (جداکننده ستون‌ها Tab باشد؛ مقادیر AGE در مثال صرفاً نمونه هستند):

```
PID PPID    AGE STATE       NAME
1   0       66  sleeping    init
2   1       65  sleeping    sh
13  2       0   running     ps
```

نکته: هسته نمی‌تواند مستقیم به حافظه فضای کاربر بنویسد؛ برای انتقال داده‌ها از توابع copyout و copyin استفاده کنید.

۵. یک فراخوانی سیستمی دیگر به نام killtree پیاده‌سازی کنید:

```
int killtree(int pid);
```

این فراخوانی سیستمی فرایندی با آن pid را به همراه **تمام نوادگانش** (فرزندان، نوه‌ها و ...) می‌کشد. کشتن یعنی دقیقاً همان کاری که kill انجام می‌دهد: علامت‌گذاری فرایند و بیدار کردن آن در صورتی که در حال sleep است، تا فرایند در اولین بازگشت به فضای کاربر خارج (exit) شود. اگر فرایندی با آن pid وجود نداشت، مقدار 1- و در حالت موفقیت مقدار 0 برگردانده شود.

برای نمایش کارکرد آن در ویدیو، می‌توانید برنامه آزمایشی t_killtree را اجرا کنید یا برنامه مشابهی بنویسید که یک درخت فرایندی می‌سازد، با killtree آن را می‌کشد و با getpinfo نشان می‌دهد همه فرایندهای درخت مرده‌اند.

آزمون خودکار

همراه این تمرین پوشه hw2-tests داده شده است. آن را در ریشه مخزن xv6 کپی کنید و با دستور زیر، این آزمون را اجرا کنید:

```
python3 hw2-tests/grade-hw2.py
```

این اسکریپت برنامه‌های آزمون را نصب می‌کند، xv6 را می‌سازد، در QEMU بوت می‌کند و پنج بررسی را اجرا می‌کند. خروجی نهایی باید score: 5/5 باشد و اسکریپت همان ابزاری است که برای نمره‌دهی استفاده می‌شود.

خروجی‌های قابل تحویل

۱. یک آرشیو فشرده (.zip) با نام hw2-lastname1-lastname2.zip شامل:
- فایل‌های تغییر داده‌شده هسته و فضای کاربر (شامل Makefile و همه فایل‌های مربوط به فراخوانی‌های سیستمی)
- کد منبع سه برنامه pingpong، race و ps
- اسکرین‌شات خروج کامل آزمون خودکار (score: 5/5)

۲. یک ویدیوی حداکثر 5 دقیقه‌ای که در آن موارد زیر را نمایش دهید:
- اجرای pingpong و گزارش نرخ تبادل
- اجرای race و توضیح اینکه چرا فرزندان برعکسِ ترتیب ساخته‌شدن جمع می‌شوند
- اجرای ps و توضیح فیلد birth که به PCB اضافه کردید و ارتباط آن با ستون AGE
- ساخت یک درخت فرایندی، اجرای killtree و نشان دادن با getpinfo که همه فرایندهای درخت مرده‌اند

در ویدیو مراحل انجام‌شده را به صورت کوتاه توضیح دهید و نیازی به توضیح خط‌به‌خط کدها نیست.

۲. لنگر

در مجموعهٔ تمرین‌های «لنگر» قصد داریم با زبان C، یک ابزار ساده برای اجرای کانتینر بسازیم و در هر مرحله، بخشی از آن را با استفاده از مفاهیم درس سیستم‌عامل توسعه دهیم. در نخستین گام، زیربنای اجرای برنامه را پیاده‌سازی می‌کنیم: دریافت یک دستور، ساخت پردازهٔ فرزند، اجرای دستور در آن و انتظار برای پایان اجرای آن. در مراحل بعد، قابلیت‌های لازم برای جداسازی محیط اجرای پردازه‌ها به همین پروژه افزوده خواهد شد.

هدف و محدودهٔ این مرحله هدف این تمرین، درک تفاوت ایجاد پردازه و اجرای برنامه، رابطهٔ پدر و فرزند و نحوهٔ دریافت وضعیت پایان اجرای فرزند است. برنامهٔ این مرحله، دستور را در محیط فعلی لینوکس اجرا می‌کند و هنوز محیط ایزولهٔ کانتینر فراهم نمی‌کند. برای انجام این تمرین صرفا نیاز به یک کرنل لینوکس که به صورت مستقیم یا Virtual Machine بالا اورده شده باشد دارید.

دریافت فایل‌های تمرین فایل‌های اولیه، راهنمای ساخت و آزمون‌های عمومی این مرحله در مخزن زیر

```
https://github.com/MohammadAminKoohi/OS-Practical
```

رابط خط فرمان نام فایل اجرایی پروژه `langar` است. پس از ساخت برنامه با `make`، دستور دلخواه باید با قالب زیر قابل اجرا باشد:

```
./langar run -- COMMAND [ARGUMENT ...]
./langar run -- /bin/echo "Hello OS"
./langar run -- printf '%s\n' 'one argument with spaces'
./langar run -- /bin/sleep 2
```

وجود جداکنندهٔ `--` الزامی است. نخستین آرگومان پس از آن، نام برنامه یا مسیر فایل اجرایی و آرگومان‌های بعدی، ورودی‌های همان برنامه هستند. نام برنامه باید هم با مسیر مشخص و هم با جست‌وجو در متغیر محیطی `PATH` پشتیبانی شود.

پوستهٔ فراخوان، آرگومان‌ها را پیش از اجرای لنگر از یکدیگر جدا کرده است؛ بنابراین نباید آن‌ها را به یک رشته تبدیل و دوباره تجزیه کنید. برای نمونه، در مثال دوم، عبارت دارای فاصله تنها یک آرگومان است. آرگومان خالی نیز باید حفظ شود. نویسه‌هایی مانند نقطه‌ویرگول که داخل یک آرگومان قرار گرفته‌اند، دادهٔ معمولی هستند و نباید توسط لنگر به‌عنوان دستور پوسته تفسیر شوند.

بخش اول: ایجاد فرزند و اجرای برنامه در فایل‌های اولیه، بخش پردازش خط فرمان در اختیار شما قرار می‌گیرد. وظیفهٔ شما تکمیل تابع زیر در فایل `src/run.c` است:

```
int run_command(char *const command_argv[]);
```

۱. با استفاده از `()fork`، یک فرزند مستقیم برای پردازهٔ لنگر ایجاد کنید. در این مرحله، برای اجرای هر دستور تنها همین یک فرزند را ایجاد کنید؛ پردازه‌هایی که خودِ برنامهٔ اجراشده می‌سازد، مشمول این محدودیت نیستند.

۲. در فرزند، برنامهٔ درخواستی را با `()execvp` و با همان آرگومان‌های دریافتی اجرا کنید. اگر اجرا موفق باشد، ادامهٔ کد فرزند پس از این فراخوان اجرا نخواهد شد.

۳. پردازهٔ پدر باید تا پایان اجرای فرزند زنده بماند و با `()waitpid` وضعیت پایان آن را دریافت کند. استفاده از حلقهٔ انتظار فعال برای بررسی پایان فرزند مجاز نیست. اگر انتظار با خطای `EINTR` قطع شد، آن را دوباره انجام دهید.

۴. ورودی استاندارد، خروجی استاندارد، خروجی خطا، متغیرهای محیطی و پوشهٔ کاری باید از لنگر به برنامهٔ اجراشده به ارث برسند. برای مثال، برنامه‌ای که از ورودی استاندارد می‌خواند، باید همچنان بتواند ورودی دریافت کند.

برای تحقق هدف آموزشی این تمرین، استفاده از `()system`، `()popen` و `()posix_spawn` مجاز نیست. همچنین نباید به‌جای اجرای مستقیم برنامه، آن را داخل یک پوسته با `sh -c` اجرا کنید.

بخش دوم: وضعیت پایان و مدیریت خطا رفتار برنامه را مطابق قواعد زیر کامل کنید:

- اگر فرزند به‌صورت عادی پایان یافت، کد خروج لنگر باید برابر کد خروج فرزند باشد؛ برای مثال، اگر برنامهٔ اجراشده مقدار 42 را برگرداند، لنگر نیز باید با همان مقدار پایان یابد.
- اگر فرزند بر اثر یک سیگنال پایان یافت، کد خروج لنگر را برابر `128 + signal_number` قرار دهید. برای استخراج وضعیت پایان، مستندات ماکروهای `WIFEXITED`، `WEXITSTATUS`، `WIFSIGNALED` و `WTERMSIG` را مطالعه کنید. در این مرحله، پیاده‌سازی رسیدگی‌کنندهٔ سیگنال یا ارسال سیگنال از پدر به فرزند خواسته نشده است.
- اگر برنامه یافت نشد، یعنی `()execvp` با خطای `ENOENT` یا `ENOTDIR` شکست خورد، کد خروج 127 باشد. برای سایر خطاهای این فراخوان، مانند نداشتن مجوز اجرا، کد خروج 126 باشد.
- در صورت شکست `()fork` یا شکست نهایی `()waitpid`، پیام خطای مناسب چاپ کنید و مقدار 1 را برگردانید. برای آزمودن این حالت‌ها، منابع سیستم را عمداً اشباع نکنید.
- برای خط فرمان نامعتبر، مانند حذف نام دستور یا جداکننده، راهنمای کوتاه استفاده چاپ شود و کد خروج 2 باشد. این رفتار در بخش پردازش خط فرمانِ فایل‌های اولیه در نظر گرفته می‌شود.
- پس از شکست اجرای برنامه در فرزند، پیام خطا را چاپ کنید و با `()_exit` پایان دهید. فرزند نباید وارد مسیر اجرای پدر شود.

تمام پیام‌های خطا و راهنمای استفاده باید در خروجی خطای استاندارد `stderr` چاپ شوند. در اجرای موفق، لنگر نباید پیام اضافی، از جمله شناسهٔ پردازه یا پیام شروع و پایان، به خروجی دستور اضافه کند. متن دقیق پیام‌های خطای سیستم ملاک ارزیابی نیست؛ مشخص بودن علت خطا و رعایت کد خروج اهمیت دارد.

آزمون و ارزیابی مبنای ارزیابی عملکرد پیاده‌سازی، نتیجهٔ آزمون‌های داور خودکار روی کد تحویلی شما است. آزمون‌ها برای قابلیت‌های مستقل، امتیازهای جداگانه دارند؛ پس درست بودن یک مثال یا نمایش اجرای موفق در ویدیو، جایگزین عبور از آزمون‌های آن قابلیت نیست. وزن آزمون‌ها در راهنمای آزمونِ منتشرشده همراه تمرین مشخص می‌شود. آزمون‌های ارزیابی فقط رفتارهای تصریح‌شده در این صورت سؤال را می‌سنجند، اما ورودی آن‌ها ممکن است با مثال‌های عمومی متفاوت باشد.

آزمون‌های عمومی، مواردی مانند اجرای دستور، حفظ آرگومان‌ها، جست‌وجوی برنامه، وراثت محیط و ورودی و خروجی، کد خروج و مدیریت خطا را بررسی می‌کنند. همچنین با یک برنامهٔ کمکی، بررسی می‌شود که دستور در فرزندی با شناسه‌ای متفاوت از لنگر اجرا شود و پدر تا پایان کار آن منتظر بماند. صرفاً درست بودن متن خروجی برای قبولی کافی نیست. استفاده از توابع خواسته‌شده و دریافت وضعیت فرزند، در کد شما نیز بررسی خواهد شد. گزارش و ویدیو برای بررسی توضیح شما و رعایت الزامات پیاده‌سازی استفاده می‌شوند؛ بنابراین تحویل آن‌ها، حتی در صورت عبور از همهٔ آزمون‌ها، الزامی است. فایل‌های آزمون یا گزارش موفقیتِ ارسالی شما، مرجع نمره‌دهی داور نیستند؛ داور نسخهٔ آزمون‌های درس را روی برنامهٔ ساخته‌شده از کد شما اجرا می‌کند.

شناسهٔ پردازه‌ها و ترتیب زمان‌بندی را ثابت فرض نکنید. برنامه باید با استاندارد C11 و گزینه‌های زیر بدون هشدار ساخته شود:

```
-Wall -Wextra -Werror
```

پرسش‌های گزارش به پرسش‌های زیر کوتاه و با ارجاع به پیاده‌سازی خود پاسخ دهید:

۱. مقدار بازگشتی `fork()` در پدر، فرزند و هنگام خطا چه تفاوتی دارد؟ هر مسیر در کد شما چه کاری انجام می‌دهد؟

۲. تفاوت `fork()` و خانوادهٔ `exec` چیست؟ پس از اجرای موفق `execvp()` چه اتفاقی برای برنامهٔ فرزند و شناسهٔ پردازهٔ آن می‌افتد؟

۳. چرا پدر باید وضعیت پایان فرزند را دریافت کند؟ اگر فرزند تمام شود اما پدرِ زنده این وضعیت را دریافت نکند، چه اتفاقی می‌افتد؟

۴. کدام ویژگی‌های لازم برای یک کانتینر هنوز در نسخهٔ فعلی لنگر وجود ندارند؟ دو مورد نام ببرید؛ پیاده‌سازی آن‌ها در این مرحله لازم نیست.

ویدیوی پاسخ و توضیح کد یک ویدیو با تصویر خوانای صفحه ضبط کنید؛ زمان پیشنهادی آن حدود پنج تا هشت دقیقه است. ویدیو باید مربوط به همان نسخه‌ای باشد که تحویل می‌دهید.

- مسیر اجرای پدر و فرزند، محل فراخوان‌های `fork()`، `execvp()` و `waitpid()`، رسیدگی به شکست اجرا و استخراج کد خروج را در کد خود نشان دهید و علت انتخاب‌های مهم را توضیح دهید؛ خواندن خط‌به‌خط کل برنامه لازم نیست.
- برنامه را از روی کد بسازید و یک اجرای موفق، یک دستور ناموجود و اجرای دستوری با کد خروج غیرصفر را نشان دهید. در هر مورد، با `echo $?` کد خروج را بلافاصله پس از پایان لنگر نمایش دهید. سپس نتیجهٔ اجرای آزمون‌های عمومی را نشان دهید؛ اگر آزمونی ناموفق است، همان نتیجه را توضیح دهید.

علاوه بر توضیح کد، به هر دو پرسش زیر در ویدیو پاسخ دهید:

۱. فرض کنید برنامه‌نویسی بدون ساخت فرزند، مستقیماً در خود پردازهٔ لنگر، `execvp()` را فراخوانی کرده است. چرا ممکن است خروجی یک دستور ساده همچنان درست به نظر برسد؟ با بررسی `PID` و `PPID` چگونه این پیاده‌سازی را از پیاده‌سازی خواسته‌شده تشخیص می‌دهید؟

۲. اجرای یک دستور که خودش با کد 7 تمام می‌شود، چه تفاوتی با شکست `execvp()` دارد؟ مسیر اجرای کد خود را در هر حالت نشان دهید و توضیح دهید چرا هنگام شکست اجرا در فرزند از `_exit()` استفاده کرده‌اید.

موارد لازم برای تحویل پاسخ این سؤال را در یک پوشه q2 تحویل دهید؛ کنید. پوشه‌ی q2 وباید مستقیماً در ریشهٔ آرشیو یا فایل فشرده شما قرار داشته باشند؛ آن‌ها را داخل یک پوشهٔ اضافی با نام دانشجو، نام مخزن یا نام تمرین قرار ندهید. ساختار بخش مربوط به این سؤال دقیقاً به شکل زیر باشد:

```
q2/
  Makefile
  src/
    main.c
    run.c
  include/
    runner.h
  answers.md
  test-results.txt
  video.mp4
```

فایل‌های اولیه را همراه تغییرات خود تحویل دهید، نه فقط فایل تغییریافته یا وصلهٔ آن را. نام فایل‌ها و کوچک و بزرگ بودن حروف آن‌ها را رعایت کنید. فایل‌های کمکیِ کد را در صورت نیاز داخل /src و /include قرار دهید.

- answers.md: پاسخ شماره‌گذاری‌شدهٔ پرسش‌های گزارش به فارسی و زمان شروع بخش‌های مهم و پاسخ دو پرسش ویدیو، مانند 02:10. در این فایل، نسخهٔ محیط اجرا و محدودیت‌های شناخته‌شدهٔ پیاده‌سازی را نیز کوتاه ذکر کنید.
- test-results.txt: خروجی متنی آخرین اجرای آزمون‌های عمومی روی همین نسخه؛ خروجی خطا را نیز ثبت کنید.
- video.mp4: فایل خودِ ویدیو با قالب MP4؛

نکته: در صورت زیاد شدن حجم ویدیو می‌توانید با ابزار ffmpeg حجم آن را کاهش داده و ورژن کاهش داده شده را قرار دهید.

اجرای داور خودکار داور پس از استخراج هر آرشیو در پوشه‌ای مستقل، از ریشهٔ آن، ساخت و اجرای برنامه را با قالب زیر انجام می‌دهد:

```
make -C q2 clean
make -C q2 CC=gcc
./q2/langar run -- COMMAND [ARGUMENT ...]
```

بنابراین هدف پیش‌فرض Makefile باید فایل اجرایی q2/langar را بسازد و هدف clean، حتی اگر هنوز فایلی ساخته نشده است، با موفقیت پایان یابد. متغیر CC را پشتیبانی کنید و گزینه‌های ساخت خواسته‌شده را حفظ کنید. دستورهای ساخت باید بدون دریافت ورودی، اتصال شبکه، نصب بسته یا دسترسی root کار کنند و در صورت شکست، وضعیت خروج غیرصفر بدهند. همهٔ کدهای لازم برای ساخت باید در همین پوشه موجود باشند؛ مسیر مطلقِ متعلق به رایانهٔ خود را در ساخت یا اجرا استفاده نکنید.

پوشهٔ کاری برنامه، همان پوشه‌ای است که داور از آن لنگر را فراخوانی می‌کند و لزوماً /q2 نیست. برنامه نباید هنگام شروع، پوشهٔ کاری را به محل فایل اجرایی تغییر دهد. مقادیر شناسهٔ پردازه‌ها، ورودی آزمون و مسیر پوشهٔ استخراج را ثابت فرض نکنید. برای هر آزمون مهلت اجرا در نظر گرفته می‌شود و برنامه نباید پس از پایان کار فرزند بی‌دلیل منتظر بماند.

کد و فایل‌های متنی را با کدگذاری UTF-8 تحویل دهید. فایل اجرایی، فایل‌های میانیِ ساخت، پوشهٔ git. و نسخهٔ محلی آزمون‌های درس را در آرشیو نگذارید. داور کد را دوباره می‌سازد؛ وجود فایل اجرایی آماده، جایگزین ساخت موفق نیست. اگر ساختار مسیرها یا رابط اجرای برنامه با این قرارداد متفاوت باشد، آزمون‌ها امکان اجرا نخواهند داشت.

۳. پردازه‌ها در لینوکس

در این سؤال، با اجرای یک دستور ساده، شناسهٔ پردازه، رابطهٔ پدر و فرزند و وضعیت پردازه را در لینوکس مشاهده می‌کنیم. برای انجام تمرین، یک محیط لینوکس روی سیستم اصلی یا Virtual Machine و ابزار ps کافی است.

بخش اول: اجرای پردازه و مشاهدهٔ مشخصات آن در یک پایانهٔ Bash، دستورهای زیر را به‌ترتیب اجرا کنید. تمام مراحل را در همان پایانه انجام دهید:

```
sleep 120 &
child=$!
echo "Shell PID: $$"
echo "Child PID: $child"
ps -o pid,ppid,stat,comm -p "$$,$child"
```

دستور sleep برای مدت مشخصی منتظر می‌ماند و سپس تمام می‌شود. قرار دادن & در انتهای دستور، آن را در پس‌زمینه اجرا می‌کند تا بتوانید پیش از پایان آن، دستورهای بعدی را وارد کنید. متغیر child شناسهٔ همین پردازه را نگه می‌دارد.

بخش دوم: بررسی اطلاعات پردازه پیش از پایان زمان انتظار، با دستور زیر اطلاعات همان فرزند را از /proc بخوانید:

```
grep -E '^(Name|State|Pid|PPid):' "/proc/$child/status"
```

مقادیر شناسهٔ پردازه و شناسهٔ پدر را با خروجی ps مقایسه کنید. سپس برای پایان دادن به همین آزمایش، دستورهای زیر را اجرا کنید:

```
kill "$child"
wait "$child"
```

دستور kill در اینجا به پردازهٔ ساخته‌شدهٔ خودتان درخواست پایان می‌دهد و wait وضعیت پایان آن را دریافت می‌کند. ممکن است پوسته پیام پایان پردازه را نمایش دهد و wait مقدار غیرصفر برگرداند؛ این نتیجه در این آزمایش طبیعی است. اگر پردازه پیش از تکمیل مشاهده‌ها به‌طور عادی تمام شد، آزمایش را از ابتدا تکرار کنید؛ شناسهٔ اجرای قبلی را دوباره استفاده نکنید.

پرسش‌های گزارش خروجی دستورها را ثبت کنید و به دو پرسش زیر کوتاه پاسخ دهید:

۱. تفاوت PID و PPID چیست؟ با استفاده از خروجی خود مشخص کنید پدر پردازهٔ sleep کدام است و آیا دو روش مشاهده، شناسه‌های یکسانی نشان می‌دهند؟

۲. وضعیت مشاهده‌شده برای sleep چیست؟ توضیح دهید این پردازه منتظر چه چیزی است و این وضعیت به کدام حالت از مدل پردازه در درس مربوط می‌شود.

ویدیوی پاسخ و توضیح دستورها یک ویدیو با تصویر خوانای صفحه ضبط کنید؛ زمان پیشنهادی آن حدود دو تا سه دقیقه است. اجرای آزمایش و خروجی‌های خود را نشان دهید و کاربرد $$ و $! را توضیح دهید. علاوه بر توضیح دستورها، به هر دو پرسش زیر در ویدیو پاسخ دهید:

۱. اگر دو نمونه از دستور sleep را هم‌زمان اجرا کنیم، آیا این دو اجرا یک پردازه محسوب می‌شوند؟ کدام مشخصات آن‌ها می‌تواند یکسان باشد و کدام شناسه باید متفاوت باشد؟ اجرای آزمایش دوم لازم نیست.

۲. اگر علامت & را از دستور اول حذف کنیم، چه تغییری در امکان وارد کردن دستور بعدی در همان پایانه ایجاد می‌شود؟ این تفاوت را با اجرای پیش‌زمینه و پس‌زمینه توضیح دهید.

آزمون و ارزیابی در این سؤال، ساختار تحویل و وجود فایل‌های لازم به‌صورت خودکار بررسی می‌شود. ارزیابی پاسخ‌ها بر اساس خروجی واقعی آزمایش، توضیح کوتاه گزارش و پاسخ‌های ویدیو است.

موارد لازم برای تحویل پاسخ این سؤال را در پوشهٔ q3، مستقیماً در ریشهٔ آرشیو یا فایل فشردهٔ تمرین و کنار پوشهٔ q2 قرار دهید. ساختار این بخش به شکل زیر باشد:

```
q3/
  answers.md
  video.mp4
```

- `answers.md`: نسخهٔ هسته با خروجی `uname -r`، دستورها و خروجی‌های آزمایش، پاسخ دو پرسش گزارش به فارسی و زمان شروع پاسخ دو پرسش ویدیو، مانند 01:10. همین فایل برای ثبت شواهد کافی است.
- `video.mp4`: فایل ویدیوی پاسخ با قالب MP4.

نکته: در صورت زیاد شدن حجم ویدیو می‌توانید با ابزار ffmpeg حجم آن را کاهش دهید و نسخهٔ کم‌حجم را قرار دهید.